Pointfree CoEndoYoneda Lemma in Functional Categories Theory and Formalization in Lean 4

Luc Duponcheel Antigravity AI

July 21, 2026

Abstract

We present a variation of the Yoneda Lemma for functional categories equipped with a monadic structure, which we formally identify as *Functional Categories*. The central theoretical contribution, the Pointfree CoEndoYoneda Lemma, establishes a natural bijection between natural transformations mapping out of a co-Yoneda endofunctor and corresponding global elements within the monadic functional category. In addition to outlining the mathematical foundations, we detail the complete formalization of this result in the Lean 4 theorem prover.

We discuss the technical challenges of working with functor compositions under strict definitional equality and demonstrate strategies for navigating dependent type theory to construct fully formal, pointfree proofs of higher-level category theory results.

1 Introduction

The *CoYoneda Lemma* serves as a cornerstone of modern category theory, asserting that for any category $\mathcal{C}$, object $X \in \mathcal{C}$, and functor $F : \mathcal{C} \to \mathbf{Set}$, the set of natural transformations $\mathrm{Nat}(\mathcal{C}(X, -), F)$ is isomorphic to the evaluation $F(X)$. This result fundamentally states that an object X is completely determined by the morphisms out of it.

In computational settings, we frequently encounter categories equipped with additional structure—specifically, categories that natively interpret types (or sets) and possess a monad-like mechanism for handling effects. We define such a category as a *Functional Category*. In this paper, we explore how the Yoneda principle extends to this structural setting.

The primary contribution is the *CoEndoYoneda Lemma*, which characterizes natural transformations that map out of an endofunctor defined via the co-Yoneda embedding. We state two main lemmas, showing that such natural transformations are entirely determined by their evaluation at the identity

morphism. We have a type-level equivalence between natural transformations from the endo-coyoneda embedding and global elements. The equivalence is, replacing elements by global elements, entirely pointfree.

We also provide a comprehensive discussion of how these concepts and theorems are formalized in the Lean 4 proof assistant using the `Mathlib` library.

2 Functional Categories

Let $\mathcal{C}$ be a locally small category. We denote the category of types (or sets) as **Type**. For any object $X \in \mathcal{C}$, the covariant Hom-functor, which we will call the co-Yoneda functor, is defined as:

$$CYF_X : \mathcal{C} \to \mathbf{Type}, \quad Y \mapsto \mathcal{C}(X, Y)$$

Definition 2.1. *A **Functional Category** is a category $\mathcal{C}$ equipped with the following data:*

1. *A designated functor $FF : \mathbf{Type} \to \mathcal{C}$.*
2. *A constructed endofunctor $GF : \mathcal{C} \to \mathcal{C}$ defined by $GF(X) = FF(CYF_{FF(1)}(X))$.*
3. *A monadic structure over GF, provided by a unit natural transformation $\gamma\eta : Id_{\mathcal{C}} \to GF$ and a multiplication natural transformation $\gamma\mu : GF \circ GF \to GF$.*

These structural components must satisfy the standard monad left unit law:

$$\gamma\eta_{GF(X)} \circ \gamma\mu_X = id_{GF(X)}$$

For now no other extra laws besides, of course, the naturality laws, are needed in the formal Lean 4 proof. This is an interesting observation on its own.

This structure essentially allows $\mathcal{C}$ to reflect the category of types into itself, while treating GF as a monadic context. To generalize Yoneda, we define the *CoYoneda EndoFunctor* $CYEF_X : \mathcal{C} \to \mathcal{C}$ for a fixed object X as the composition:

$$CYEF_X = CYF_X \circ FF$$

2.1 Formalization of Functional Categories in Lean 4

In Lean 4, this structure is elegantly captured using a type class extending the standard `LargeCategory` from `Mathlib`.

```
class FunctionalCategory (C : Type (u + 1)) extends LargeCategory.{u} C where
  FF : Type u ⥤ C

  γμ : GF_def FF ⋙ GF_def FF ⟶ GF_def FF
  γη : 1 _⟶ GF_def FF

  γ_left_unit :
    ∀ (X : C), γη.app ((GF_def FF).obj X) ≫ γμ.app X = id ((GF_def FF).obj X)

  φ {X Y : Type u} (f : X → Y) : FF.obj X⟶FF.obj Y := FF.map (TypeCat.ofHom f)

  φ_eq :
    ∀ {X Y : Type u} (f : X → Y), φ f = FF.map (TypeCat.ofHom f) :=
      by intros; rfl

  γη_φ :
    ∀ (W : Type u), γη.app (FF.obj W) = φ (fun (w : W) ⇒ φ (fun (_ : PUnit) ⇒ w)) :=
      by intros; rfl
```

This class natively encapsulates the natural transformations and the left unit law constraint. A canonical instance of this class is **Type** itself, where FF is the identity functor.

```
instance typesFunctionalCategory : FunctionalCategory (Type u) where
  FF := 1 (Type u)
  γη := {
    app := fun X ⇒ (1 (Type u)).map (↑(fun (x : X) ⇒ ↑(fun _ : PUnit ⇒ x)))
    naturality := fun X Y f ⇒ by ext; rfl
  }
  γμ := {
    app := fun X ⇒ (1 (Type u)).map (↑(fun (f : PUnit⟶PUnit⟶X) ⇒ ↑(fun p : PUnit
          ⇒ (f p) p)))
    naturality := fun X Y f ⇒ by ext; rfl
  }
  γ_left_unit := fun X ⇒ by rfl
```

3 The CoEndoYoneda Lemmas

The core results consist of two lemmas showing how natural transformations out of $CYEF_Z$ uniquely correspond to global elements.

3.1 CoEndoYoneda Lemma 1

Lemma 3.1. *For any monadic functional category $\mathcal{C}$, endofunctor $F : \mathcal{C} \to \mathcal{C}$, and object $Z \in \mathcal{C}$, there is an injection mapping natural transformations $\tau : CYEF_Z \to F$ into transformations targeting $F \circ GF$.*

We define `globalEndoTransformation` which takes τ and post-composes it with the monad unit $\gamma\eta$. Lemma 1 asserts that this transformation is identical to the one constructed by taking the evaluation of τ at the identity morphism 1_Z and lifting it through FF.

```
def globalFunctorialValueToNaturalTransformation1
  (F : C ⥤ C) {X : C} (g : FF.obj PUnit⟶F.obj X) :
    CY\!EF X⟶F ⋙ GF where
  app Y :=
    φ ((fun f ⇒ g ≫ F.map f) : (X⟶Y) → (FF.obj PUnit⟶F.obj Y))
  -- (naturality proof omitted)

theorem coEndoYonedaLemma1 {F : C ⥤ C} {Z : C} (τ : CY\!EF Z⟶F) :
    globalEndoTransformation τ =
    globalFunctorialValueToNaturalTransformation1 F
      ((φ (X := PUnit) (Y := Z⟶Z) (fun _ ⇒ id Z)) ≫ τ.app Z)
```

3.2 CoEndoYoneda Lemma 2

Lemma 2 is the more substantial result, demonstrating a complete bijection when the target functor is $F \circ GF$.

Theorem 3.2. *Let $\tau : CYEF_Z \to F \circ GF$ be a natural transformation. Then τ is uniquely determined by its evaluation at 1_Z. There exists a bijection between such transformations and global elements $g : FF(1) \to GF(F(Z))$.*

In Lean, we construct the transformation out of the global element g and then prove that any arbitrary τ is exactly this constructed transformation:

```
theorem coEndoYonedaLemma2 {F : C ⥅ C} {Z : C} (τ : CY\!EF Z ⟶ F ⋙ GF) :
    τ = globalFunctorialValueToNaturalTransformation2 F
      ((φ (X := PUnit) (Y := Z ⟶ Z) (fun _ ⇒ id Z)) ≫ τ.app Z) := by
  ext Y
  -- ... (proof steps)
```

3.3 EndoCoYoneda Equivalence

Building upon Lemma 2, we can establish a formal equivalence between the set of natural transformations out of the co-Yoneda endofunctor and the corresponding global elements. This result is the direct generalization of the classical CoYoneda equivalence to our monadic functional category setting.

Theorem 3.3. *There is a type-level equivalence between natural transformations $CYEF_X \to F \circ GF$ and elements of $FF(1) \to GF(F(X))$.*

In Lean 4, we define this as an `Equiv` (denoted by $\simeq$).

Utilizing `coEndoYonedaLemma2` enables us to effortlessly prove the left inverse property. For the right inverse, we rely on specialized pointfree mapping lemmas that handle constant functions evaluated under φ:

```
def endoCoYonedaEquiv {F : C ⥅ C} {X : C} :
    (CY\!EF X ⟶ F ⋙ GF) ≃ (FF.obj PUnit ⟶ GF.obj (F.obj X)) where
  toFun τ := (φ (X := PUnit) (Y := X ⟶ X) (fun _ ⇒ id X)) ≫ τ.app X
  invFun g := globalFunctorialValueToNaturalTransformation2 F g
  left_inv := fun τ ⇒ (coEndoYonedaLemma2 τ).symm
  right_inv := fun g ⇒ by
    exact φ_const_comp_γμ g
```

This equivalence provides a completely pointfree and logically verified dictionary for evaluating and constructing natural transformations in functional categories.

4 Proof Techniques and Challenges in Lean 4

Formalizing abstract category theory in dependent type theory (DTT) introduces unique difficulties, particularly concerning definitional equality.

4.1 Strict Functor Composition

In standard mathematics, functor composition is strictly associative. In Lean 4, however, these are represented as distinct abstract syntax trees. They are *isomorphic* via `Category.assoc`, but they are not *definitionally equal*.

When writing the proof of `coEndoYonedaLemma2`, this limitation prevents standard term-rewriting tactics like `simp` from effortlessly reducing deeply nested compositions.

4.2 Constructing Equality Chains

To circumvent the strict type-checker without resorting to manual casts (such as `eqToHom`), the proof for `coEndoYonedaLemma2` abandons the monolithic `simp` approach in favor of explicit equality chains acting on the natural transformation components.

By utilizing `Eq.trans` and `congr_arg`, we explicitly guide the type checker through the unit laws and naturality squares:

```
have h1 :
  τ.app Y =
    τ.app Y ≫ id ((GF_def FF).obj (F.obj Y)) :=
      (Category.comp_id (τ.app Y)).symm
have h2 :
  id ((GF_def FF).obj (F.obj Y)) =
    γη.app ((GF_def FF).obj (F.obj Y)) ≫ γμ.app (F.obj Y) :=
      (γ_left_unit (F.obj Y)).symm
have h_left_unit :
  τ.app Y =
    τ.app Y ≫ (γη.app ((GF_def FF).obj (F.obj Y)) ≫ γμ.app (F.obj Y)) :=
      Eq.trans h1 (congr_arg (fun g ⇒ τ.app Y ≫ g) h2)
```

This localized approach bypasses the rigid typing of the $\longrightarrow$ category morphisms by acting directly on the equality type `=` in the universe of types. This technique is highly robust and avoids the infamous "motive is not type correct" errors common in Lean category theory formalization.

5 Conclusion

The Pointfree CoEndoYoneda Lemma is a variation of the classical Pointful CoYoneda Lemma in a monadic functional category setting. By successfully formalizing the lemma in Lean 4, we have not only verified its absolute logical correctness but also established a concrete methodology for handling strict composition equality in dependent type theory. The approach of combining explicit structural definitions with targeted, step-by-step equality chaining provides a reliable method for formalizing advanced category theory in modern proof assistants.